

Matplotlib & Data Visualization

What is Data Visualization?

Data Visualization is the graphical representation of data using charts and graphs. It helps us identify patterns, trends, comparisons, distributions, and relationships in data.

Common Uses:

- Sales Analysis
- Business Reports
- Stock Market Analysis
- Customer Analytics
- Scientific Research
- Machine Learning

1. Line Plot

Definition

A Line Plot displays data points connected by straight lines.

When to Use

- Sales over time
- Website visitors per day
- Temperature changes
- Stock prices

Syntax

```
plt.plot(x, y)
```

Important Attributes

color

Controls line color.

Possible Values:

- red

- blue
- green
- black
- #FF5733

Example:

```
plt.plot(x, y, color="red")
```

Effect:

Changes the line color.

marker

Displays symbols at data points.

Possible Values:

Marker	Shape
o	Circle
s	Square
^	Triangle
*	Star
x	Cross
D	Diamond

Example:

```
plt.plot(x, y, marker="o")
```

Effect:

Each data point becomes visible.

linestyle

Controls line style.

Possible Values:

Value	Style
-	Solid
--	Dashed
:	Dotted
-.	Dash Dot

Example:

```
plt.plot(x, y, linestyle="--")
```

linewidth

Controls thickness.

Common Values:

- 1 → Thin
- 2 → Normal
- 3 → Thick
- 5 → Very Thick

Example:

```
plt.plot(x, y, linewidth=3)
```

alpha

Controls transparency.

Range:

- 0.0 → Invisible
- 0.2 → Very Transparent
- 0.5 → Semi Transparent
- 1.0 → Fully Visible

Example:

```
plt.plot(x, y, alpha=0.5)
```

label

Used for legends.

Example:

```
plt.plot(x, y, label="Sales")  
plt.legend()
```

Example: Monthly Sales

Python

```
import matplotlib.pyplot as plt  
  
months = ["Jan", "Feb", "Mar", "Apr", "May"]
```

```
sales = [20000, 25000, 22000, 30000, 35000]

plt.plot(months, sales,
         color="blue",
         marker="o",
         linestyle="-",
         linewidth=2)

plt.title("Monthly Sales")
plt.xlabel("Months")
plt.ylabel("Sales")
plt.grid(True)
plt.show()
```

2. Bar Chart

Definition

Represents data using rectangular bars.

When to Use

- Product Sales Comparison
- Employee Performance
- Monthly Revenue
- Brand Comparison

Syntax

```
plt.bar(x, y)
```

Important Attributes

color

Controls bar color.

Example:

```
plt.bar(x, y, color="red")
```

width

Controls bar thickness.

Common Values:

- 0.3 → Thin
- 0.5 → Medium
- 0.8 → Thick
- 1.0 → Maximum Width

Example:

```
plt.bar(x, y, width=0.8)
```

edgecolor

Adds border around bars.

Example:

```
plt.bar(x, y, edgecolor="black")
```

alpha

Controls transparency.

Example:

```
plt.bar(x, y, alpha=0.7)
```

label

Used for legends.

Example:

```
plt.bar(x, y, label="Sales")
```

Example: Students in Different Departments

Python

```
import matplotlib.pyplot as plt

departments = ["CSE", "ECE", "EEE", "MECH"]

students = [120, 95, 80, 60]

plt.bar(departments, students,

        color="green",

        edgecolor="black",

        width=0.6)

plt.title("Department Strength")

plt.xlabel("Department")

plt.ylabel("Students")

plt.show()
```

3. Histogram

bins

Controls the number of intervals (groups).

Common Values:

- 5 → Less Detail
- 10 → Standard
- 20 → More Detail
- 50 → Very Detailed

Example:

```
plt.hist(data, bins=10)
```

Effect:

- More bins → More detailed distribution
- Fewer bins → Simpler distribution

color

Controls histogram color.

Example:

```
plt.hist(data, color="green")
```

edgecolor

Adds border to each bin.

Example:

```
plt.hist(data, edgecolor="black")
```

alpha

Controls transparency.

Example:

```
plt.hist(data, alpha=0.7)
```

histtype

Controls histogram style.

Possible Values:

- bar (Default)
- step
- stepfilled

Example:

```
plt.hist(data, histtype="step")
```

density

Converts frequencies into probabilities.

Possible Values:

- True
- False

Example:

```
plt.hist(data, density=True)
```

Example: Exam Scores

```
Python
import matplotlib.pyplot as plt

marks =
[45, 56, 67, 78, 89, 90, 55, 60, 61, 70, 72, 85, 92, 68, 73, 81, 65, 74, 77, 88]

plt.hist(marks,
         bins=5,
         color="orange",
         edgecolor="black")

plt.title("Exam Score Distribution")
plt.xlabel("Marks")
plt.ylabel("Students")
plt.show()
```

4. Scatter Plot

Definition

A Scatter Plot displays the relationship between two numerical variables using points.

When to Use

- Advertising Spend vs Sales
- Study Hours vs Marks
- Height vs Weight
- Temperature vs Ice Cream Sales
- Experience vs Salary

Syntax

```
plt.scatter(x, y)
```

Important Attributes

s

Controls marker size.

Common Values:

- 20 → Small
- 50 → Normal
- 100 → Large
- 300 → Very Large

Example:

```
plt.scatter(x, y, s=100)
```

c

Controls marker color.

Example:

```
plt.scatter(x, y, c="red")
```

Can also accept numeric values.

Example:

```
plt.scatter(x, y, c=sales)
```

cmap

Color map used when c contains numeric values.

Popular Values:

- viridis
- plasma
- coolwarm
- inferno
- jet
- Blues
- Reds

Example:

```
plt.scatter(x, y, c=sales, cmap="plasma")
```

alpha

Controls transparency.

Range:

- 0.0 to 1.0

Example:

```
plt.scatter(x, y, alpha=0.8)
```

marker

Controls marker shape.

Possible Values:

Marker	Shape
o	Circle
s	Square
^	Triangle
*	Star
x	Cross
D	Diamond
+	Plus
p	Pentagon

Example:

```
plt.scatter(x, y, marker="D")
```

edgecolors

Adds border color.

Example:

```
plt.scatter(x, y, edgecolors="black")
```

linewidths

Controls border thickness.

Example:

```
plt.scatter(x, y, linewidths=2)
```

Example: Study Hours vs Marks

Python

```
import matplotlib.pyplot as plt

hours = [1,2,3,4,5,6,7,8]

marks = [30,40,50,60,70,80,90,95]

plt.scatter(hours, marks,

            s=100,

            c="red",

            marker="o")

plt.title("Study Hours vs Marks")

plt.xlabel("Study Hours")

plt.ylabel("Marks")

plt.show()
```

5. Pie Chart

Definition

A Pie Chart displays proportions or percentages as slices of a circle.

When to Use

- Market Share
- Budget Allocation
- Department Expenses
- Customer Segments
- Browser Usage

Syntax

```
plt.pie(values)
```

Important Attributes

labels

Adds names to slices.

Example:

```
plt.pie(values, labels=brands)
```

autopct

Displays percentages.

Common Values:

- %1.0f%%
- %1.1f%%
- %1.2f%%

Examples:

%1.0f%% → 25%

%1.1f%% → 25.4%

%1.2f%% → 25.43%

startangle

Rotates the chart.

Common Values:

- 0
- 90
- 180
- 270

Example:

```
plt.pie(values, startangle=90)
```

explode

Separates slices.

Examples:

[0,0,0]

No separation

```
[0,0.1,0]
```

Second slice slightly separated

```
[0,0.3,0]
```

Second slice highly separated

Example:

```
plt.pie(values, explode=[0,0.2,0])
```

shadow

Adds shadow effect.

Possible Values:

- True
- False

Example:

```
plt.pie(values, shadow=True)
```

colors

Custom colors for slices.

Example:

```
plt.pie(values,  
colors=["red","green","blue"])
```

Example: Market Share

Python

```
import matplotlib.pyplot as plt  
brands = ["Apple", "Samsung", "OnePlus", "Others"]  
share = [35, 30, 20, 15]  
plt.pie(share,  
        labels=brands,  
        autopct="%1.1f%%",  
        startangle=90,  
        shadow=True)  
plt.title("Mobile Market Share")  
plt.show()
```

6. Box Plot

Definition

A Box Plot displays data distribution, median, quartiles, spread, and outliers.

When to Use

- Exam Score Analysis
- Salary Comparison
- Outlier Detection
- Data Cleaning
- Performance Analysis

Syntax

```
plt.boxplot(data)
```

Important Attributes

labels

Labels for datasets.

Example:

```
plt.boxplot(data,  
labels=["Class A","Class B"])
```

vert

Controls orientation.

Possible Values:

- True → Vertical
- False → Horizontal

Example:

```
plt.boxplot(data, vert=False)
```

notch

Displays notches around median.

Possible Values:

- True
- False

Example:

```
plt.boxplot(data, notch=True)
```

patch_artist

Allows box filling.

Possible Values:

- True
- False

Example:

```
plt.boxplot(data,  
patch_artist=True)
```

showmeans

Displays mean value.

Possible Values:

- True
- False

Example:

```
plt.boxplot(data,  
showmeans=True)
```

meanline

Shows mean as line.

Possible Values:

- True
- False

Example:

```
plt.boxplot(data,  
meanline=True)
```

boxprops

Customizes box appearance.

Example:

```
boxprops=dict(linewidth=2)
```

medianprops

Customizes median line.

Example:

```
medianprops=dict(linewidth=3)
```

flierprops

Customizes outliers.

Example:

```
flierprops=dict(marker="o")
```

Example: Employee Salaries

Python

```
import matplotlib.pyplot as plt

salary = [25000, 27000, 30000, 32000, 35000,
          36000, 38000, 40000, 42000, 70000]

plt.boxplot(salary,
            showmeans=True,
            patch_artist=True)

plt.title("Salary Distribution")

plt.show()
```


7. Area Chart

Definition

An Area Chart is a Line Chart with the area below the line filled with color.

When to Use

- Revenue Growth
- Website Traffic
- Population Growth
- Sales Trends

Syntax

```
plt.fill_between(x, y)
```

Important Attributes

color

Controls fill color.

Example:

```
plt.fill_between(x,y,  
color="skyblue")
```

alpha

Controls transparency.

Range:

- 0.0 to 1.0

Example:

```
plt.fill_between(x,y,  
alpha=0.5)
```

label

Used in legends.

Example:

```
plt.fill_between(x,y,  
label="Visitors")
```

linewidth

Controls line thickness when combined with plot().

Example:

```
plt.plot(x,y,  
linewidth=2)
```

Example:

```
plt.plot(x,y,  
linestyle="--")
```

Example: Website Visitors

Python

```
import matplotlib.pyplot as plt  
  
days = [1,2,3,4,5,6,7]  
  
visitors = [200,250,230,280,300,320,350]  
  
plt.fill_between(days, visitors,  
                 color="skyblue",  
                 alpha=0.5)  
  
plt.plot(days, visitors,  
         color="blue",  
         linewidth=2)  
  
plt.title("Website Visitors")  
plt.xlabel("Days")  
plt.ylabel("Visitors")  
  
plt.show()
```

8. Heatmap

Definition

A Heatmap represents data using colors. Higher values and lower values are shown using different color intensities, making it easy to identify patterns and trends.

When to Use

- Temperature Analysis
- Student Attendance Reports
- Correlation Matrix
- Sales Performance Dashboard
- Website Activity Analysis
- Employee Productivity Reports

Syntax

```
sns.heatmap(data)
```

Important Attributes

data

The 2-dimensional data to visualize.

Examples:

- List of Lists
- NumPy Array
- Pandas DataFrame

Example:

```
sns.heatmap(data)
```

annot

Displays values inside cells.

Possible Values:

- True
- False

Example:

```
sns.heatmap(data, annot=True)
```

Effect:

Shows exact values inside each box.

fmt

Controls value formatting.

Common Values:

Value	Output Example
d	25
.1f	25.4
.2f	25.43
.3f	25.432

Example:

```
sns.heatmap(data,  
annot=True,  
fmt=".2f")
```

cmap

Controls color theme.

Popular Color Maps:

Colormap	Appearance
YlOrRd	Yellow → Orange → Red
Blues	Light Blue → Dark Blue
Reds	Light Red → Dark Red
Greens	Light Green → Dark Green
viridis	Purple → Green → Yellow
plasma	Purple → Orange

coolwarm	Blue → Red
inferno	Black → Orange
magma	Black → Pink

Example:

```
sns.heatmap(data,
cmap="coolwarm")
```

linewidths

Controls cell border thickness.

Common Values:

- 0
- 0.5
- 1
- 2

Example:

```
sns.heatmap(data,
linewidths=0.5)
```

xticklabels

Labels for X-axis.

Example:

```
xticklabels=["Mon","Tue","Wed"]
```

yticklabels

Labels for Y-axis.

Example:

```
yticklabels=["Week1","Week2"]
```

cbar

Displays color scale bar.

Possible Values:

- True
- False

Example:

```
sns.heatmap(data,  
cbar=True)
```

Effect:

Shows color meaning on the side.

square

Makes cells square shaped.

Possible Values:

- True
- False

Example:

```
sns.heatmap(data,  
square=True)
```

Real-Time Example

Weekly Temperature Analysis

Days:

- Monday
- Tuesday
- Wednesday
- Thursday
- Friday
- Saturday
- Sunday

Weeks:

- Week 1
- Week 2
- Week 3
- Week 4

Heatmap colors show which days are hotter and cooler.

Example: Student Marks

Python

```
import seaborn as sns

import matplotlib.pyplot as plt

data = [

    [80, 75, 90],

    [60, 70, 85],

    [95, 88, 92]

]

sns.heatmap(data,

            annot=True,

            cmap="YlOrRd",

            linewidths=1)

plt.title("Student Marks")

plt.show()
```

9. Subplots

Definition

Subplots allow multiple charts to be displayed inside a single figure window.

When to Use

- Dashboards
- Reports
- Business Analytics
- Data Comparison
- KPI Monitoring

Syntax

`plt.subplot(rows, columns, position)`

Understanding Positions

1 Row × 2 Columns

Position Layout

1 2

Examples:

`plt.subplot(1,2,1)`

`plt.subplot(1,2,2)`

2 Rows × 2 Columns

Position Layout

1 2

3 4

Examples:

`plt.subplot(2,2,1)`

`plt.subplot(2,2,2)`

`plt.subplot(2,2,3)`

`plt.subplot(2,2,4)`

3 Rows × 3 Columns

Position Layout

1 2 3

4 5 6

7 8 9

Examples:

`plt.subplot(3,3,5)`

Displays chart in center position.

Important Attributes

figsize

Controls overall figure size.

Syntax:

```
plt.figure(figsize=(width,height))
```

Examples:

```
plt.figure(figsize=(8,6))
```

```
plt.figure(figsize=(10,8))
```

```
plt.figure(figsize=(15,10))
```

Effect:

Larger figure gives more space for charts.

tight_layout()

Automatically adjusts spacing between charts.

Example:

```
plt.tight_layout()
```

Effect:

Prevents chart overlap.

suptitle()

Adds title for entire dashboard.

Example:

```
plt.suptitle("Sales Dashboard")
```

Effect:

Single heading for all charts.

Example: Sales Dashboard

Python

```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr"]

sales = [20, 25, 30, 35]

profit = [5, 7, 8, 10]

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)

plt.plot(months, sales, marker="o")

plt.title("Sales")

plt.subplot(1, 2, 2)

plt.bar(months, profit)

plt.title("Profit")

plt.tight_layout()

plt.show()
```

Common Plot Functions

title()

Adds chart title.

Example:

```
plt.title("Monthly Sales")
```

xlabel()

Adds X-axis label.

Example:

```
plt.xlabel("Months")
```

ylabel()

Adds Y-axis label.

Example:

```
plt.ylabel("Revenue")
```

legend()

Displays legend.

Example:

```
plt.legend()
```

grid()

Displays grid lines.

Examples:

```
plt.grid(True)
```

```
plt.grid(linestyle="--")
```

```
plt.grid(alpha=0.5)
```

show()

Displays chart.

Example:

```
plt.show()
```

Collab Link: [🔗 graphs.ipynb](#)